

第二讲：计算——Julia 与 VFI

异质性主体宏观经济学的计算方法

孙杰

2026 年 5 月 12 日

多伦多大学

准备

贝尔曼方程 (Bellman equation)

$$V(b, z) = \max_{c \in [0, b]} u(c) + \beta V(R(b - c) + z, z) \quad \forall b, z$$

此方程刻画了求解家庭问题所需的函数 V 。

V 的唯一性与可计算性

对任意有界的 v_1, v_2 ,

$$\|\mathcal{T}v_1 - \mathcal{T}v_2\|_{\infty} \leq \beta \|v_1 - v_2\|_{\infty}.$$

非正式地, Banach 不动点定理 (Banach fixed-point theorem) 意味着:

- 存在唯一的 V^* 满足 $V^* = \mathcal{T}V^*$ 。
- 值函数迭代 (value function iteration, VFI) 总能找到 V^* 。

本讲: 用有限网格代替连续状态 b , 把 $\mathcal{T}$ 写成一个 Julia 函数, 迭代到不动点, 并画出 V_0 。

Julia

数组 (Arrays)

```
xs = [1.0, 2.0, 3.0]      # Vector{Float64}, length 3
M  = [1 2; 3 4]           # Matrix{Int64}, 2×2
zs = zeros(3, 4)          # 3×4 matrix of 0.0
xs[1]                     # 1.0 --- 1-indexed!
M[2, 1]                   # 3
```

- 索引从 1 开始。
- `xs[end]` 是最后一个元素。`xs[2:3]` 是一个切片。
- 数组按列优先存储 (column-major)：维度 1 为行，维度 2 为列。“向量”即列向量。

广播 (Broadcasting)

```
xs = [1.0, 2.0, 3.0]

xs .+ 10      # [11.0, 12.0, 13.0]
log.(xs)      # [0.0, 0.693, 1.099]
xs .^ 2       # [1.0, 4.0, 9.0]

# elementwise on two arrays of the same shape:
[1,2,3] .* [10,20,30] # [10, 40, 90]
```

点号 `.` 是“逐元素” (elementwise) 的标记。

点号可让任意函数或运算按元素作用，无需循环。

函数 (Functions)

```
function utility(c)
    c <= 0 && return -Inf
    return log(c)
end

# shorthand for simple conditionals:
function utility(c)
    return c <= 0 ? -Inf : log(c)
end

# one-line functions equally valid:
u(c) = c <= 0 ? -Inf : log(c)
```


离散化

离散财富网格

- V 是定义在 $[0, \infty) \times W$ 上的函数。我们无法精确表示它。
- 选取 N 个财富取值 $b_1 < b_2 < \dots < b_N$ 。
- 把 V 表示为向量 $[V(b_1), \dots, V(b_N)]$ 。
- 这样函数就变成了有限维的。

注：值函数迭代的概念**并不依赖于** V 是否被表示为向量。 V 也可以是样条 (spline)、神经网络，或定义在动态网格上的函数，相关概念都同样适用。

线性网格 vs 对数网格

```
loggrid(lo, hi, N) = exp.(range(log(lo), log(hi); length=N))

# linear: equally spaced
b_grid = collect(range(0.1, 20.0; length=200))

# log: denser near the borrowing constraint
b_grid = loggrid(0.1, 20.0, 200)
```

策略函数（policy function）在 b 较小时非线性更强，因此我们使用对数网格，把更多网格点放在较小的 b 上。

效用函数

```
u(c) = c <= 0 ? -Inf : log(c)
```

更一般的版本是带参数 γ 的 CRRA，但目前先保持简单。

把贝尔曼算子 (Bellman operator) $\mathcal{T}$ 写成 Julia 函数

```
function bellman_operator(V, params)
    (;  $\beta$ , R, z, b_grid) = params

    # end-of-period wealth needed to reach each  $b'$  on the grid
    b_end = (b_grid' .- z) ./ R

    #  $u(c) + \beta V(b')$  for every ( $b, b'$ ); take max over  $b'$ 
    return maximum(u.(b_grid .- b_end) .+  $\beta$ .*V'; dims=2)
end
```

b_grid 是列向量, b_grid' 是行向量。

每一行对应一个 b 值, 每一列对应一个 b' 值。最后一行代码在每个 b 内 (行内)、跨 b' (列) 取 max。

关于示例代码的说明

笔记本中的代码**并未**做优化，也并不追求速度，只为说明概念。
更快的实现见 HouseholdStages.jl 软件包。

值函数迭代

值函数迭代

1. 从 $V_0 \equiv 0$ 开始（任何初始猜测均可）。
2. $V_{k+1} \leftarrow \mathcal{T}V_k$ 。
3. 当 $\|V_{k+1} - V_k\|_\infty < \text{tol}$ 时停止。

solve_vfi

```
function solve_vfi(params; tol=1e-6, maxiter=2000, verbosity=0)
    V = zeros(size(params.b_grid))

    iters,  $\Delta$  = 0, Inf
    while  $\Delta$  >= tol
        TV = bellman_operator(V, params)
        V,  $\Delta$  = TV, maximum(abs.(TV .- V))
        iters += 1
        iters > maxiter && error("VFI did not converge in $maxiter iterations")
    end

    verbosity >= 1 && println("VFI converged in $iters iterations with error ",  $\Delta$ )
    return V
end
```

基准校准 (Baseline Calibration)

```
b_grid = loggrid(0.05, 40.0, 400)
params = (; β=0.96, R=1.04, z=1.0, b_grid)

V = solve_vfi(params; verbosity=1)
```

- 需要 $\beta R < 1$, 否则 V^* 为无穷大。
- 网格: 从 0.05 到 40 的 400 个对数间距点。
- params 是一个具名元组 (NamedTuple)。末尾的 b_grid 等价于 b_grid=b_grid。

值函数的性质

V 的形状与 u 的形状相似¹:

- $V(b)$ 单调递增 (更多财富 $\Rightarrow$ 更多未来消费)。
- $V(b)$ 是凹函数: 对穷人而言, 多一单位财富比对富人更有价值。
- 斜率即**财富的边际价值**: 由包络定理 (envelope theorem), $V'(b) = u'(c^*(b))$ 。

¹Sheeran (2017)

策略函数 c^*

$$c^*(b) = \arg \max_{c \in [0, b]} u(c) + \beta V(R(b - c) + z).$$

```
function policy_function(V, params)
    (; β, R, z, b_grid) = params

    # same setup as bellman_operator
    b_end = (b_grid' .- z) ./ R

    # argmax over b' (columns) — return the chosen b'-index for each row
    idx = argmax(u.(b_grid .- b_end) .+ β.*V'; dims=2)
    return vec(getindex.(idx, 2))::Vector{Int}
end
```

与 `bellman_operator` 相同，只是把 `maximum` 换成了 `argmax`。
返回一个整数向量：每个（行） b 所对应最优 b' 的网格索引。